

EduCanvas 第二轮深度架构与代码审计

审计基线： `main@245c4262`

审计目标：从产品、Agent、数据、前端、安全、性能、测试、运维和团队治理等维度，判断 EduCanvas 当前真实成熟度。

一、我现在对 EduCanvas 的完整理解

EduCanvas 已经不是一个“AI 教学聊天网页”。

我对它的准确理解是：

EduCanvas 正在尝试建立一个以 Notebook 为长期上下文容器、以 Gateway 为逻辑控制面、以单一 Agent Runtime 为执行内核、以教育可信服务为差异化能力、以 Canvas 为统一人机协作面的个人 Agent 平台。

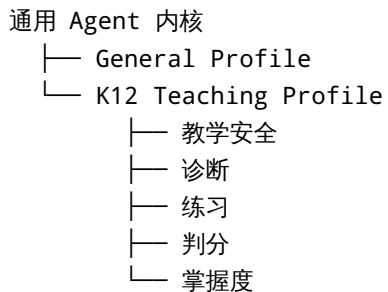
它的核心结构不是：

用户 → 聊天页面 → 调用大模型

而是：

```
用户身份
  ↓
Personal Agent
  ↓
Notebook 权限与上下文
  ↓
Gateway Operation
  ↓
TurnApplicationService
  ↓
Context Engine → Agent Loop → Tool Kernel
  ↓
Model / RAG / Artifact / Runtime / Worker
  ↓
可信事实、审计记录与可恢复终态
```

教育能力也不是第二个 Agent，而是运行在同一个 Agent 内核上的受控领域 Profile：



这一理解与仓库的产品定义和第二代架构文档一致：Web、TUI、Channel 是同一个 Agent 的入口，Notebook 聚合 Source、Conversation、Artifact 和 Memory，Gateway 提供身份与 Operation 权威，教育事实由确定性领域服务维护。

但我现在也更明确地看到：

EduCanvas 的“可信基础设施”已经相当成熟，而“产品收敛、数据单轨、运行成本、真实效果和生产治理”明显落后于基础设施复杂度。

换句话说，目前是一个**底盘比车身更成熟**的项目。

二、产品边界：技术上不是空想，但产品范围已经过宽判断

产品定位完整，但缺少真正的阶段性主产品。

产品文档同时包含：

- 通用个人 Agent ；
- K12 教学 ；
- Notebook RAG ；
- Canvas ；
- Web Runtime ；
- Python 实验 ；
- 实时语音 ；
- TUI ；
- Telegram ；
- Capability Node ；
- Personal Memory ；
- 家庭与班级协作 ；
- 教师审核 ；
- 多媒体生成 ；

- 未来微信、QQ 和其他渠道。

这些能力之间有统一的架构解释，因此不能说是毫无关系的功能堆砌。问题是：

产品定义宽到几乎所有 Agent 基础设施都可以被解释成“核心能力”。

路线图甚至同时推进 Canvas Runtime、语音、Experiment Runtime、Memory、通用知识库、Provider 配置、教师资料审核和生产治理。

风险

这会导致项目长期处于：

基础能力还有缺口

- 再补一条基础设施
- 新基础设施又需要测试、权限、协议和文档
- 用户主流程迟迟无法稳定冻结

技术团队会感觉一直在取得进展，但很难回答：

- 用户最常用的三个动作是什么？
- 哪个动作比 NotebookLM、ChatGPT 或普通 RAG 产品明显更好？
- 教育能力到底是用户选择 EduCanvas 的首要原因，还是架构上的身份标签？
- 竞赛演示完成后，哪个场景可以持续产生真实留存？

建议

未来一个阶段只允许一个主产品定义，例如：

学生上传教材或课堂资料，EduCanvas 基于可追溯证据完成讲解、生成练习、判分和下一步推荐。

其他能力必须回答“是否直接提升这一主流程”，否则降级为实验能力。

三、模块化单体：目录合理，但逻辑耦合度仍然偏高

当前优点

仓库采用：

```
apps/  
packages/
```

而不是盲目拆微服务。这是正确选择。

Agent Core、Runtime、Gateway、Teaching、DB、Canvas、Telemetry 等模块有相对明确的职责，应用层承担组合根。

真正的问题

模块数量多，不代表模块真正低耦合。

例如 Web 应用直接依赖：

- Agent Core ；
- Agent Runtime ；
- Asset Processing ；
- Canvas Protocol ；
- DB ；
- Gateway Core ；
- Gateway Runtime ；
- MCP Runtime ；
- Model Gateway ；
- Node Runtime ；
- Teaching Core ；
- Teaching Runtime ；
- Telemetry。

这说明 `apps/web` 并不是简单的 UI 加 BFF，而是几乎理解整个系统。

我的定性

EduCanvas 当前是：

目录上的模块化单体，组合根上的双核心单体。

两个核心分别是：

1. 独立 Gateway ；
2. Next.js Web Server。

它们都知道身份、路由、Agent Runtime、DB、模型和工具，只是能力集合不同。

四、多入口架构：控制语义统一了，执行装配还没有统一

这是上一轮“双轨问题”的进一步细化。

当前有三条主要 Turn 路径：

```
Gateway General
Web General
Web Teaching
```

三条路径确实都使用 `TurnApplicationService`，所以“唯一 Agent Loop”是真实事实，不是文档包装。

但三者仍然分别装配：

- Lifecycle；
- Profile；
- Repository；
- Tool Kernel；
- Cancellation；
- Model Runtime；
- Telemetry。

具体问题

1. Gateway 和 Web 能力不对称

Gateway General 目前拒绝所有非纯文本输入，包括 `asset_ref`。Web General 则支持 Asset、图像、网页和 Artifact。

因此：

```
同一个 Agent
≠ 相同能力内核
```

现在更准确的说法是：

```
共享同一 Loop 实现
+ 不同入口分别装配能力
```

2. Web 的 Gateway 兼容路径过于绕

Web 请求会经历：

```
Web Request
→ Gateway Envelope
→ Gateway Service
→ WebCompatibilityRunner
→ Turn Application Event
→ Gateway Event
→ Legacy Web Event
```

这套结构证明了协议收敛意图，却也证明 Web 仍是独立执行平台。

风险

未来接入 Memory、语音、Credential 或新 Tool 时，仍然可能出现：

- Gateway 接了，Web 没接；
- Web General 接了，Teaching 没接；
- Tool Policy 相同，但 Adapter 不同；
- 同一个 failure code 被不同入口重新翻译。

建议

不需要拆服务，而是建立唯一的组合工厂：

```
createTurnApplication({
  profile,
  entrypoint,
  capabilities,
  identity,
  persistence,
})
```

Web 和 Gateway 只能在工厂外提供传输 Adapter，不再各自重新装配完整运行时。

五、Agent Loop：可靠，但仍然是固定策略的 v1 引擎

做得好的地方

Agent Loop 对供应商事件实施了严格零信任验证：

- Schema strict parse；
- phase 校验；

- 唯一终态；
- tool call ID 唯一；
- Usage 与 metadata 一致；
- 文本和工具调用顺序约束；
- 工具数量和参数大小限制；
- 输出长度限制；
- 取消和重试。

这比大多数直接依赖 AI SDK 默认行为的 Agent 项目严谨得多。

当前限制

1. 预算是字符预算，不是 token 或成本预算

当前限制是：

```
MAX_RESPONSE_CHARACTERS = 128000
```

这并不能准确表示：

- 中文 token；
- 英文 token；
- 图片 token；
- Prompt token；
- Tool Result token；
- Provider 成本。

同样的 128K 字符，在不同模型上的成本和可用上下文差异很大。

2. 工具轮数是固定钳位

Agent Loop 将工具轮数限制在 1-4 之间。

这保证了不会无限循环，但没有按任务类型区分：

- 简单问答；
- 网页研究；
- 多来源验证；
- Artifact 生成；
- 高成本工具；
- 低成本本地工具。

3. 每轮工具结果持续累积

所有 `accumulatedResults` 会继续传入后续模型请求。如果工具返回内容接近上限，多轮后上下文会快速膨胀。

当前主要依赖 Adapter 自己做结果裁剪，没有统一的 Tool Result compaction 策略。

建议

下一代预算应该至少使用：

```
输入 token
+ 预估输出 token
+ 工具调用预算
+ 模型调用次数
+ 预估货币成本
+ 总时间预算
```

并且由 Profile 决定预算模板，而不是只有 `maxToolRounds`。

六、Context Engine：确定性强，但“选什么”仍然非常初级

当前机制

Context Engine 做的是稳定、可解释的贪心选择：

```
required 优先
→ priority 高优先
→ 原始顺序
→ maxSegments / maxCharacters 截断
```

这比完全把上下文拼起来可靠，也能记录被选中的 Message 和 Asset。

General Profile 的真实上下文策略

当前通用对话：

- 最多先取 40 条消息；
- 只保留最近 24 条已完成消息；

- 当前消息 required ；
- 历史消息 priority 递增 ；
- Asset 全部 required ；
- Memory 明确为 `not_implemented` ；
- 总预算 128K 字符。

这意味着什么

目前所谓 Context Engine，更接近：

一个有确定性审计记录的上下文裁剪器。

它还不是：

- 语义压缩系统 ；
- 长对话总结系统 ；
- 相关历史检索系统 ；
- 动态 token allocator ；
- Memory 与 Conversation 的冲突解决器。

已发现的一个具体追溯缺陷

这是本轮最明确的新代码问题之一。

多张原生图片会被合并为一个 Context Candidate ：

```
images.map(image => image data)
```

但该 Segment 的：

```
assetVersionId: images[0].versionId
```

只记录第一张图片的版本 ID。

而 Context Engine 最终的 `selectedAssetVersionIds` 是从每个 Segment 的单个 `assetVersionId` 字段生成。

因此可能出现：

实际发送给模型：4 张图片
Context Snapshot：只登记第 1 张图片版本

这不意味着其他图片绕过了前置归属验证，但会导致 **Turn Context** 审计材料不能完整重建本轮模型实际看到的图片集合。

修复建议

将：

```
assetVersionId?: string
```

升级为：

```
assetVersionIds?: readonly string[]
```

或者每张图片保留独立 Segment，同时引入 `segmentCost`，避免占满 Segment 数量。

严重度：P1。

七、RAG：数据库检索设计扎实，质量观测不足

当前强项

混合检索不是简单地把 FTS 分数和余弦分数直接相加，而是使用 RRF：

```
PostgreSQL FTS  
+  
pgvector ranking  
→ Reciprocal Rank Fusion
```

系统还确保：

- 先冻结本 Turn 可见 Source 版本；
- 向量只在同模型、版本和指令下比较；
- Chunk 哈希改变后旧向量失效；
- 向量查询 1.5 秒超时；
- 向量失败退回 FTS；
- Retriever 和版本写入候选事实；
- 候选结果进入引用白名单。

这一层的可信性很好。

问题一：错误原因被完全压平

查询 Embedding 失败时：

```
catch {  
    return null;  
}
```

调用方只能看到本次退回词法检索，却无法从返回结果区分：

- Embedding 未配置；
- Provider 超时；
- Provider 返回错误维度；
- API Key 失效；
- 网络失败；
- 模型版本错误。

对用户而言，降级是正确的；对运维而言，完全吞掉原因是不够的。

应当实现：

用户响应：仍然正常降级
内部 Trace：记录稳定 failure reason
健康状态：累积 embedding degradation 指标

问题二：检索算法强于检索评测

当前能证明：

- 查询不会越权；
- 排序确定；
- 失败可降级；
- 引用有白名单。

但不能充分证明：

- Top-k 真的相关；
- RRF 比纯 FTS 更好；
- 1.5 秒超时是否合理；
- Chunk 策略是否适合教材；
- 召回提升是否转化为回答质量提升。

项目路线图也仍把教学质量评测和真实教材评测列为后续项。

我的定性

RAG 的安全工程成熟度高于检索科学成熟度。

八、Tool Kernel：副作用纪律强，但内存幂等缓存的价值被高估

Tool Kernel 有：

- Adapter 注册；
- 多维权限；
- 参数 Schema；
- executionId；
- semantics hash；
- Tool Call Ledger；
- Effect Ledger；
- 审批；
- outcome unknown；
- reconciliation。

这些是非常好的基础。

细化问题

`ToolKernel` 内部维护最多 1024 条 execution cache。

但 Web General、Web Teaching 和 Gateway 都是在每次创建 Turn Application 时新建 `ToolKernel`。

所以这个缓存的有效范围通常只是：

单个 TurnApplication 实例

它不能提供：

- 跨请求幂等；
- 跨进程幂等；
- 服务重启后的幂等；
- Worker 续跑幂等。

真正承担持久幂等的是 Ledger，而不是这张 Map。

建议

把注释和设计说明改准确：

内存缓存：防止单实例并发重复执行
持久 Ledger：承担跨请求和恢复语义

并评估是否真的需要每个 Turn 保留 1024 条缓存。单 Turn 工具调用硬上限很低，这个默认值与真实生命周期并不匹配。

九、数据架构：数据库约束优秀，但数据访问包正在失去边界

优点

迁移中有大量数据库级约束，而不是只依赖 TypeScript：

- 外键；
- 组合外键；
- 状态 CHECK；
- 终态一致性 CHECK；
- 唯一索引；
- 所有权索引；
- Hash 格式约束。

例如 Web Runtime Run 的状态和终态字段由数据库 CHECK 共同约束。

这是真正的工程质量。

问题一：@educanvas/db 是超级公共入口

该包同时导出：

- `getDb`；
- 全部 Schema；
- 数十个 Repository；
- 多套聊天和 Turn 仓储；
- RAG；
- Gateway；
- Audio Retention；
- Runtime；

- Security Audit ;
- Pagination ;
- Dual Write。

这让 DB 包成为事实上的第二个 `agent-core` : 什么都知道。

问题二：迁移数量增长非常快

Drizzle Journal 已经有大量连续迁移，主干至少推进到了 `0047`。

快速演进本身没问题，但自动生成的名称如：

```
0047_illegal_dorian_gray
```

对实际发布审查没有语义价值。

未来出现生产迁移事故时，运维人员看到的不是：

```
0047_add_web_runtime_runs
```

而是无意义的随机名称。

建议

1. 将 schema 和底层 DB 访问移出默认出口；
 2. 为迁移增加语义化说明文件；
 3. 每个迁移记录：
 4. 是否锁表；
 5. 是否重写大表；
 6. 是否可回滚；
 7. 预计耗时；
 8. 兼容部署顺序；
 9. production 前建立 expand/migrate/contract 纪律。
-

十、认证与 Web 安全：实现靠谱，但生产安全依赖部署环境

已确认的强项

Session：

- 使用 32 字节随机 Token；
- 数据库只保存 SHA-256 Hash；
- Cookie 为 HttpOnly；
- 生产环境使用 `__Host-` 前缀；
- SameSite Lax；
- Secure；
- 撤销后删除 Cookie。

写请求执行同源校验，JSON 响应默认 no-store、nosniff 并附带 request ID。

登录错误也不会区分用户名不存在和密码错误。

当前限制

生产限流并不在仓库内实现。

非本地环境必须设置：

```
EDUCANVAS_AUTH_RATE_LIMIT_MODE=shared-upstream
```

否则登录直接返回 503。

这个 fail-closed 行为是正确的，但也说明：

当前“认证安全可生产”依赖一个尚未由仓库提供或验证的上游基础设施。

同源检查还依赖反向代理正确处理：

- Host；
- X-Forwarded-Proto。

这不是直接漏洞，但生产部署文档必须明确要求入口代理清洗这些 Header，不能接受客户端任意透传。

十一、Canvas：统一的是资源协议，还不是完整渲染实现

Canvas 的统一可分为三层：

统一资源协议
统一 Renderer Registry
统一真实渲染链

当前前两层基本成立，但第三层尚未完全成立。

具体事实

Source PDF、图片、文本、DOCX、音频和视频已经通过 Registry Renderer 渲染。

但 Artifact 的 Registry Renderer 目前仍统一指向：

ArtifactCompatibilityRenderer

其行为不是渲染 Artifact，而是显示：

请通过受控的 Artifact 详情入口打开此内容。

虽然 Registry 中已经登记了 mind map、slides、flashcards、note 等类型，但实际渲染仍然交给旧 `ArtifactCanvas` 路径。

我的定性

Canvas 的资源发现和打开协议已经统一，但 Artifact 渲染内核仍然双轨。

因此“统一 Canvas 已完成”更准确的表述应该是：

统一资源访问和选择边界已完成，
Artifact 实际渲染迁移尚未完成。

十二、前端架构：设计系统在改善，但 Workspace 状态管理已经到临界点

GeneralChatWorkspace 同时控制：

- 对话；
- Composer；
- Sources；
- Studio；
- Artifact；
- Canvas；
- HTML Preview；
- Sidebar；
- Landing；
- Session Storage 恢复；
- 在线状态；
- GSAP；
- Accessibility；
- 错误状态。

这已经构成一个没有显式状态机的复杂状态机。

具体风险

当前可能同时存在：

```
studioOpen
sourceResource
previewHtml
artifactFlow.openDetail
resource pending
canvasSelected
assetPanel
```

这些状态理论上存在互斥关系，但主要由各个 handler 手动清理：

```
setSourceResource(null)
setPreviewHtml(null)
artifactFlow.closeCanvas()
```

任何一个新增入口漏掉一行清理，就可能出现：

- 多个层同时打开；
- 旧 Preview 残留；
- Artifact 和 Source 状态冲突；
- 全屏与分栏行为不同。

错误语义也不一致

例如 Artifact 列表加载失败被转成：

```
setStudioItems([])
```

用户看到的是“没有产物”，而不是“产物加载失败”。

Asset 刷新失败也有直接吞掉的路径。

这与后端强调的“诚实失败”形成反差：

后端拒绝把失败伪装为空结果，前端部分路径却仍然这么做。

建议

不要只拆 JSX，应建立显式 Workspace 模式：

```
type WorkspaceSurface =  
  | { type: 'none' }  
  | { type: 'studio' }  
  | { type: 'source'; resourceId: string }  
  | { type: 'artifact'; artifactId: string }  
  | { type: 'html'; source: string }  
  | { type: 'loading'; target: ... }  
  | { type: 'failed'; target: ...; error: ... };
```

这样互斥关系由类型保证，而不是靠几十个 `setState(null)`。

十三、测试体系：数量真实，但测试数字容易造成错误安全感

强项

CI 实际包含：

- Gitleaks；
- Lint；
- Typecheck；
- Unit；
- PostgreSQL Integration；
- Windows Smoke；
- Runtime Pressure；
- Production Build；
- Runtime Composition；
- Browser E2E。

这明显高于普通项目。

需要降温的地方

1. 默认浏览器覆盖只有 Chromium Desktop

Playwright 只配置了：

```
Desktop Chrome  
workers = 1
```

没有：

- WebKit；
- Firefox；
- Mobile Chrome；
- 触摸交互；
- 不同屏幕尺寸。

2. UI 检查被明确排除

默认 CI 使用：

```
grepInvert: /@ui/
```

视觉、响应式、焦点和动效交给单独人工路线。

这对于减少抖动是合理的，但意味着默认 CI 不能证明：

- Canvas 没有布局破坏；
- 移动端可用；
- 焦点顺序正确；
- GSAP 状态转换正确；
- Design Token 使用一致。

3. CI 允许一次重试

同时设置：

```
failOnFlakyTests = true  
retries = 1
```

这可以发现 flaky test，但“测试最终通过”仍可能是第二次才通过。

建议在 CI 输出中单独汇总 retry count，不要只看绿色状态。

4. 没有看到覆盖率门禁

Web Vitest 配置没有 coverage threshold，根 CI 也没有 coverage job。

测试数量很多，但可能集中在：

- Schema；
- Failure code；
- Repository contract；
- 小型纯函数。

而复杂 UI Controller、跨入口行为和实际模型质量可能覆盖不足。

十四、可观测性：已经有 Trace Adapter，但还没有生产闭环

Telemetry Runtime 已经支持：

- OTLP HTTP Exporter ；
- Batch Span Processor ；
- Sampling ；
- Exporter 降级 ；
- Health Snapshot ；
- Force Flush ；
- Shutdown。

Web 也通过全局单例避免开发热重载重复注册 Provider。

当前不足

它目前主要是 Trace，而不是完整 Observability：

Trace：有
Metrics：未看到完整运行闭环
Structured logs：不统一
SLO：未完成
Alert：未完成
Collector：部署外部依赖
Dashboard：未完成
Runbook：未证明

路线图也明确把外部 Collector、SLO、告警、备份和恢复演练列为 production 前置项。

更深一层的问题

很多内部降级被设计为“不影响用户”：

- Embedding 失败 → FTS ；
- Telemetry 失败 → NOOP ；
- Node capability 查询失败 → 空集合 ；
- Artifact 列表失败 → 空数组。

这些都提高了可用性，但如果没有指标，系统会长期处于“看起来可用，实际大量降级”的状态。

必须建立：

```
degradation rate
fallback rate
provider error rate
retrieval vector applied rate
tool outcome unknown rate
artifact failure rate
```

十五、性能与成本：有硬上限，但没有系统化成本模型

当前已经有不少局部保护：

- Context 128K 字符；
- 输出 128K 字符；
- Tool 参数 64KB；
- Tool 数量限制；
- 模型重试限制；
- Vector 查询超时；
- Runtime 资源限制；
- Worker 批量上限。

这些能防止明显失控。

但没有看到完整的：

```
每用户成本
每 Notebook 成本
每 Turn 模型调用次数
Prompt token
Completion token
Embedding 成本
Artifact 生成成本
工具成本
运行时 CPU 时间
```

路线图也把 Gateway、模型、工具和 Runtime 的成本配额列为 production 门槛。

可能的隐形成本来源

1. 每个复杂 Turn 最多四轮工具加一次 Synthesis；
2. 历史消息和所有 required Asset 进入 Context；
3. Model 配置在同一请求路径中可能多次解析；
4. 图片输入整体切换到视觉模型；

- 5. 查询 Embedding 在教学检索时同步调用；
- 6. 前端依赖 Three.js、GSAP、Shader、PDF 等大型库。

CI 中没有看到：

- Bundle size budget；
- Route payload budget；
- Web Vitals gate；
- Provider cost regression gate。

十六、部署与运维：本地开发成熟，生产部署仍是空位

当前 Docker Compose 只负责 PostgreSQL。

Web、Gateway、Worker 等仍由本地主机上的 orchestrator 启动。Makefile 对本地开发非常友好，提供：

- doctor；
- setup；
- dev；
- TUI；
- integration；
- E2E；
- DB migration。

这说明什么

EduCanvas 目前的运维成熟度是：

本地可重复开发：较好
CI 验证：很好
单机演示：较好
真实生产部署：尚未定义完整方案

缺口包括：

- Web/Gateway/Worker 部署拓扑；
- Database Backup/PITR；
- 对象存储；
- Secret Manager；
- Rolling migration；
- Health Probe；
- Autoscaling；
- Queue backlog 告警；

- 灾难恢复；
- 数据导出、修正与删除；
- Upstream rate limit；
- OTLP Collector。

这些缺口仓库自己也有诚实记录，因此不能称为架构失败，只能说明 production readiness 仍然有限。

十七、供应链与依赖治理：有锁文件和 Secret Scan，但还不够严格

CI 使用：

```
actions/checkout@v4
pnpm/action-setup@v4
actions/setup-node@v4
gitleaks/gitleaks-action@v2
```

而不是完整 commit SHA。

对于普通项目很常见，但 EduCanvas 已经把自己定位为未成年人教育与可信 Agent 平台，供应链要求应该更高。

建议 production 前增加：

- GitHub Actions SHA pinning；
- Dependency Review；
- 自动依赖更新；
- License policy；
- SBOM；
- Container image digest；
- Runtime 审计依赖白名单；
- 模型文件 Manifest 与来源证明；
- Release provenance。

语音模型 Manifest 的校验思路已经是一个好起点。

十八、运行版本一致性：存在明确的 Node 类型错位

仓库运行基线是：


```
Node 22
```

但多个包使用：

```
@types/node ^26.1.1
```

例如 Web 和 Gateway。

这会让 TypeScript 允许使用 Node 26 API，但实际运行在 Node 22。

它不是理论架构问题，而是一个应立即修正的工程配置问题。

严重度：P1，修复成本极低。

十九、文档与 Agent 开发流程：非常先进，也非常危险

当前 workflow 实际上是：

```
负责人制定原子任务
→ DeepSeek 实现
→ Codex 审计
→ 负责人决定 PASS / BLOCK
→ PR 合并
→ 台账回写
```

UV 计划甚至明确规定了：

- 文件所有权；
- 任务依赖；
- 每次只能领取一个任务；
- 验证命令；
- Diff discipline；
- Agent 完成报告格式。

这是一套很先进的 AI 协作开发系统。

风险一：文档成为代码之外的第二套运行时

Agent 是否能正确修改项目，越来越依赖：

- AGENTS ；
- CLAUDE ；
- Active Plan ；
- ADR ；
- Evidence Ledger ；
- File Ownership ；
- Task Unlock 状态。

一旦文档漂移，Agent 可能严格执行错误指令。

风险二：审查角色并不真正独立

CODEOWNERS 当前所有路径都指向 @Timcai06 。

最新 PR #284 没有 requested reviewer，PR 讨论也为空。

因此“DeepSeek 实现、Codex 审核”在 GitHub 证据层面仍然可能表现为：

同一负责人身份下的自动化自审

风险三：计划状态已经漂移

任务线文件已经推进到 V12，但 Active README 一度仍写 V09/V15 已解锁。这证明人工维护多个状态源必然会漂移。

建议

将任务状态机器化：

```
id: V15
status: pass
implementation_pr: 284
review_commit: 9cd25cc
evidence:
  - worker integration
  - db integration
```

README 和依赖图自动生成，Agent 不直接编辑重复状态表。

二十、最终成熟度判断

当前最成熟的部分

1. 失败关闭；
2. Agent 协议验证；
3. 数据库约束；
4. Tool 副作用纪律；
5. 权限和 Notebook 归属意识；
6. 单元与集成测试；
7. Runtime 隔离意识；
8. 证据和审计文化。

当前中等成熟的部分

1. Web UI 架构；
2. Canvas 统一；
3. RAG 质量；
4. 多入口一致性；
5. Model Provider 组合；
6. 账号系统；
7. Telemetry。

当前仍不成熟的部分

1. 产品范围收敛；
2. 真实教学质量；
3. 长期 Memory；
4. 正式 IdP；
5. 多实例部署；
6. 成本与配额；
7. SLO 与告警；
8. 数据治理；
9. 独立代码审查；
10. 用户试点和效果证据。

二十一、这一轮最值得立即修复的具体问题

P1：明确代码缺陷或高价值修复

1. 多张原生图片只记录第一张 `assetVersionId`；

2. Node 22 与 `@types/node` 26 不一致；
3. Tool 名称和错误码仍有从展示文案推断的兼容逻辑；
4. `@educanvas/db` 默认导出 `getDb` 和全部 Schema；
5. Web/Gateway/Teaching 分别装配 Turn Application；
6. Active 计划状态存在漂移；
7. 前端失败被部分转换为空结果。

P2：架构收敛

1. 平台 Turn 与旧 K12 持久事实单轨化；
2. Artifact Renderer 真正迁入统一 Registry；
3. General Workspace 改为显式 UI 状态机；
4. Context 改为 token/cost 预算；
5. RAG 增加离线评测和降级指标；
6. Tool Kernel 明确内存缓存与持久幂等的不同职责。

P3：生产治理

1. 多浏览器和移动端 E2E；
2. Bundle/Web Vitals 门禁；
3. Metrics、SLO 和告警；
4. Backup/PITR 演练；
5. 共享限流；
6. Supply-chain hardening；
7. 数据导出、更正与删除；
8. 独立 Review 身份。

二十二、最核心的结论

EduCanvas 不是一个“代码质量不足，需要重构”的项目。

它现在真正面临的是：

基础设施复杂度已经逼近一个小型商业平台，但产品验证、团队规模和生产治理仍停留在优秀个人项目阶段。

下一阶段再继续扩展更多能力，技术上依然可以做得很好，但项目的总质量未必继续上升。

接下来最正确的动作不是：

继续增加模块、协议、测试和任务线

而是：

删除双轨
压缩概念
统一组合根
补真实评测
完成一条用户闭环
证明可部署、可观测、可维护

我对 EduCanvas 当前阶段的最终判断是：

它已经证明了你能构建可信 Agent 基础设施；接下来必须证明你能控制复杂度，并把这套基础设施转化为真正清晰、稳定、有效的教育产品。